

DressMe

Phase 4 — Full prototype

AI personal fashion assistant · ESPRIT, Advanced Data Science project (team of 6) · Report generated 2026-10-04

Executive summary

- Phase 4 turns the Phase 3 dataset (*dressme.csv*, 3 public sources, unified schema) into a working app, built as **five sub-projects**: FashionCLIP embeddings, an EfficientNet classifier, a compatibility formula, a FastAPI + MongoDB backend and a React frontend. All five are done.
- **FashionCLIP** gives a 512-number vector for 282,135 pictures / crops. A linear probe on these vectors reaches 88.3% category and 79.7% sub_category: the bar the classifier had to beat.
- **EfficientNet-B0** (three heads) beats it on every field: category **95.7%**, sub_category **86.4%**, pattern **86.8%** on the test split. The app uses it for all three.
- The **compatibility formula** (style + colour + pattern + structure, weights set by the team) scores outfits 0-100 with reasons. Swap-test AUC: 66.6% Polyvore, 78.7% Fashionpedia. Style carries almost all the measurable signal.
- The **backend** exposes wardrobe upload with automatic analysis, "should I buy this?", outfit suggest / score / complete, look-alikes in the wardrobe and the H&M shop catalogue, a chat assistant with function calling (Gemini, or a local model with Ollama) and an admin dashboard.
- The **frontend** (React 19 + Tailwind 4) covers scan, wardrobe, outfit building, look-alikes, chat and profile in English, French and Arabic (RTL).
- **Real photos**: uploads are turned upright (phone EXIF rotation) and their background is removed (U2-Net, plus cloth-seg when the item is worn), so they look like the white product shots the models were trained on. Outfits never repeat a sub_category or exceed a category limit.
- **Demo readiness**: 50 backend tests pass; a browser walkthrough of every screen and the follow-up work found and fixed three bugs before the jury demo (*DEMO.md*). The pipeline for our own test photos is ready (*LOCAL_PHOTOS.md*); collection is next.
- **Local chat model** (in progress): the assistant can run on our own machine with Ollama (Qwen3-4B, no key, no 20-requests-a-day quota, works offline). A synthetic dataset in English, French and Darija and a QLoRA fine-tuning pipeline on Kaggle are ready; the base model's baseline is measured, training is next.

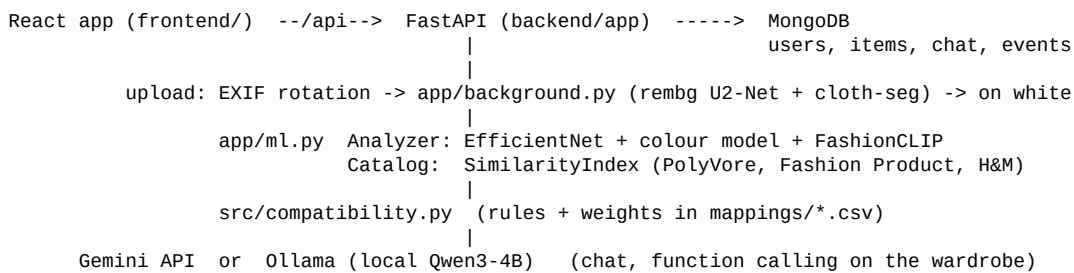
1. Context and goals

Target users are young Tunisians on a limited budget who buy mostly second-hand (friperie): items are often unlabelled and can rarely be returned, so every purchase is a small gamble. The Phase 1 survey (57 responses) set the priorities: 42% want "should I buy this?" advice, 56% shop second-hand, black dominates 84% of wardrobes.

Phase 4 goal: a full prototype that (1) digitises a wardrobe from photos, (2) answers buy / think / skip from one photo in the shop, (3) suggests and scores outfits filtered by modesty, season and occasion, (4) finds look-alikes, and (5) offers a chat assistant that only talks about clothes the user owns.

Sub-project	Output	Status
4.1 Embeddings	FashionCLIP vectors + similarity index	done
4.2 Classifier	EfficientNet-B0: category, sub_category, pattern	done
4.3 Compatibility	weighted formula, 0-100 score with reasons	done
4.4 Backend	FastAPI + MongoDB + Gemini chat	done
4.5 Frontend	React + Tailwind app and admin dashboard	done
4.6 Local chat	Ollama engine + QLoRA fine-tuning (Qwen3-4B)	in progress

2. Architecture



Models train on the local GPU (RTX 2050, 4 GB). The backend reuses the *src/* code directly, so the evaluated models are exactly those the app runs. Team-set rules (weights, colour harmony, pattern mixing, outfit structure) live in CSV files, never in code.

3. FashionCLIP embeddings

- src/embed_fashionclip.py*: one 512-d unit vector (float16) per picture, i.e. per *image_group* for product shots and per item crop (≥ 32 px) for Fashionpedia. 282,135 vectors; resumable, one shard per dataset.
- src/similarity.py* (*SimilarityIndex*): nearest-neighbour search, reused by the API for look-alikes and for the H&M shop.
- src/evaluate_embeddings.py* $\rightarrow$ *reports/embeddings_evaluation.md*.

Results on the 29,633 test pictures

Task	Fashion Product	PolyVore	Fashionpedia	All
Zero-shot category	83.6%	80.8%	62.2%	72.2%
Zero-shot sub_category	70.7%	71.5%	50.4%	60.4%
Linear probe category	96.3%	94.8%	80.8%	88.3%
Linear probe sub_category	88.1%	93.6%	69.1%	79.7%
Linear probe pattern	72.4%	—	74.4%	74.2%

- Colour**: on hand-labelled items the vectors do worse than the Phase 3 gradient-boosting model (PolyVore 63.1% vs 76.7%, Fashionpedia 42.4% vs 65.9% at the 0.7 cut), so the gradient-boosting colour model stays.

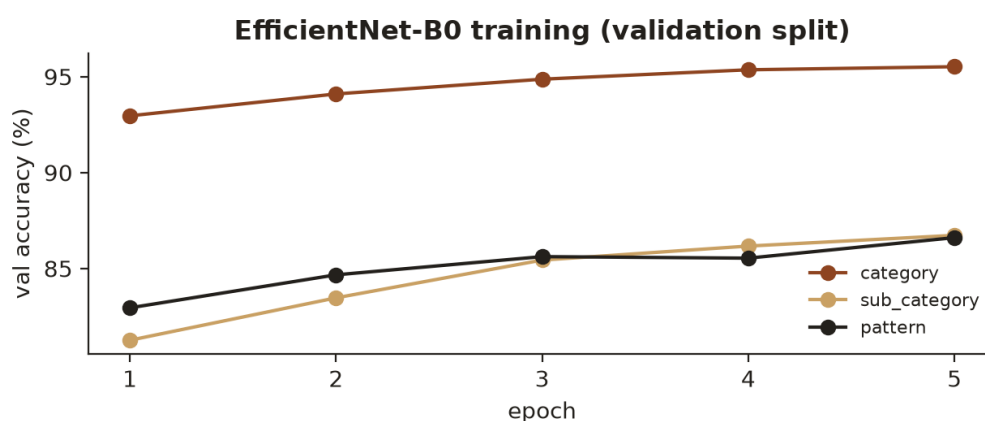
- **Street → shop retrieval:** for 300 Fashionpedia crops, 65% of the 5 nearest product shots share the street item's category.
- The probes are trained on the GPU (*Probe* class): sklearn's logistic regression was far too slow on this machine.



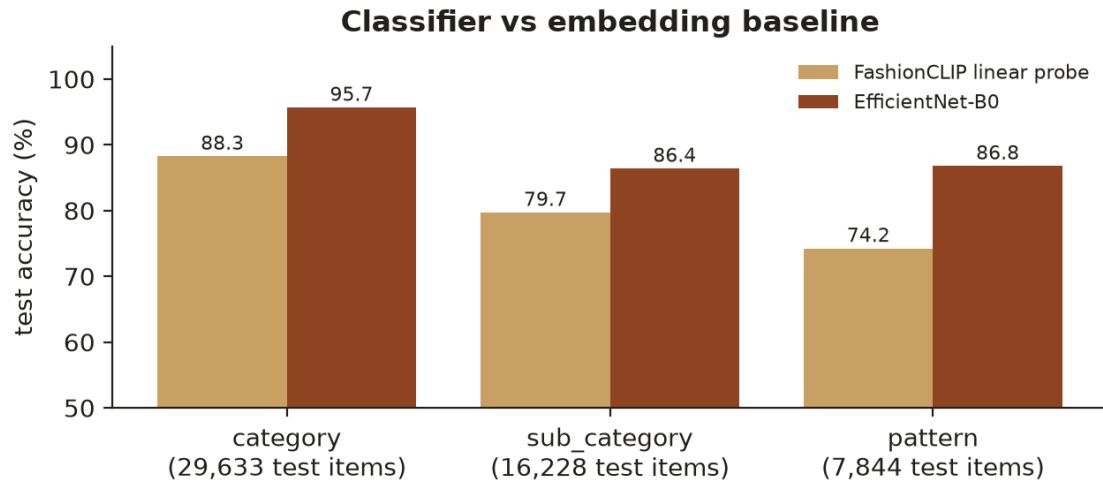
Street photo crops (left) and their nearest product shots.

4. EfficientNet classifier

- EfficientNet-B0 with ImageNet weights and **three heads** (category, sub_category, pattern). Each picture only trains the heads it has a label for; classes are weighted $1/\sqrt{\text{freq}}$.
- `src/build_image_cache.py` packs every picture, letterboxed to a 224 px white square, into one file: opening 282k small files on Windows ran at ~300 img/s.
- `gpu_batch` decodes JPEGs and augments on the GPU (no hue change, so colours stay true). Training: 5 epochs, ~20 min each (~102 min total) at ~200 img/s, resumable.
- `predict` keeps the sub_category inside the predicted category, as the app shows it.



Validation accuracy per epoch (reports/classifier_training_log.csv); the best epoch is the last one.

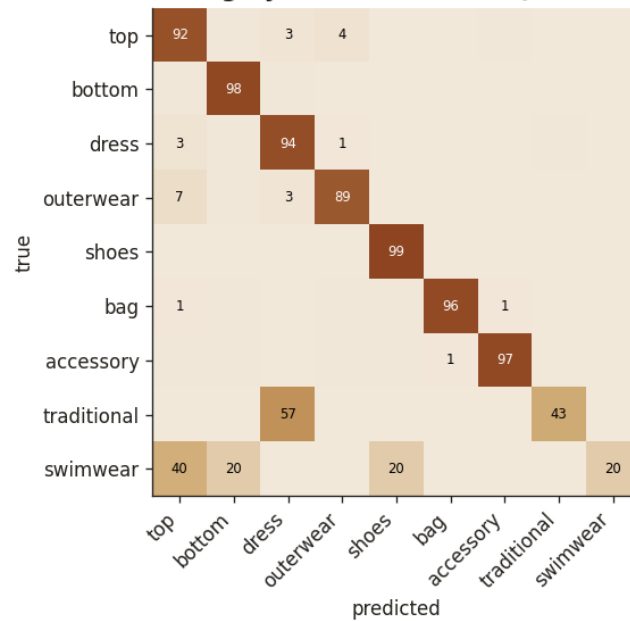


Same 29,633 test pictures for both models. Team rule: the app uses whichever is better per field.

Field	Fashion Product	PolyVore	Fashionpedia	All	Balanced
category	98.6%	97.7%	93.2%	95.7%	80.9%
sub_category	89.2%	94.3%	81.2%	86.4%	79.8%
pattern	82.9%	—	87.1%	86.8%	77.0%

The biggest gain is on Fashionpedia street photos (category 93.2% vs 80.8%, sub_category 81.2% vs 69.1%): fine-tuning learns to read crops of worn clothes. **Traditional** (3/7 right, mostly read as dress) and **swimwear** (1/5) are too small to learn, which explains the lower balanced accuracy.

EfficientNet: category confusion on test (% of each true class)



Category confusion matrix on the test split.

5. Compatibility formula

src/compatibility.py: score = weighted sum of four parts from 0 to 1, shown as 0-100 with reasons. A part that cannot be computed (e.g. no colour known) is left out and the other weights are rescaled.

Part	Weight	How it is computed
style	0.35	mean FashionCLIP similarity between items, rescaled 0.20 → 0, 0.80 → 1
colour	0.30	colour-group pair table (colour_harmony.csv) + penalty above 3 bold colours
pattern	0.15	worst clothing pattern pair (pattern_mixing.csv); predictions under 0.6 confidence ignored
structure	0.20	top + bottom or a full piece, plus shoes and per-category limits (outfit_structure.csv)

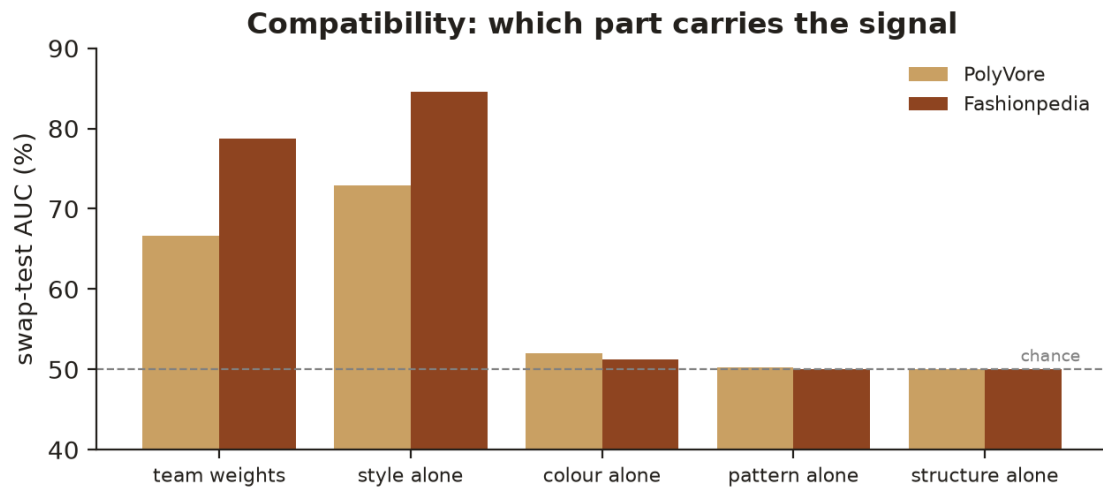
- **Functions:** *filter_items* (profile: min_coverage, season, occasion; unknown fields pass), *suggest_outfits*, *complete_outfit* and *buy_advice*.
- **Hard rule (clashes):** an outfit never holds the same sub_category twice (two pairs of jeans) or more items of a category than *outfit_structure.csv* allows. *complete_outfit* never proposes a clash, and scoring one answers 422.
- **Should I buy this?** Counts the good outfits (score ≥ 60) where the new item beats every owned item of its category: ≥ 5 → buy, ≥ 1 → think, else skip. It also flags near-twins already in the wardrobe (similarity ≥ 0.9).
- **Weights and rules are set by the team;** the data only measures them. They are still marked REVIEW until the team tunes them, and admins can edit them live from the dashboard.

Evaluation

AUC = how often a real outfit beats the same outfit with one item swapped for a random item of the same category (50% = chance). FITB = finding the hidden item among 4 of the same category (25% = chance).

Weights	PolyVore AUC	PolyVore FITB	Fashionpedia AUC	Fashionpedia FITB
team weights	66.6%	42.6%	78.7%	59.5%
style alone	72.9%	49.7%	84.6%	70.0%
colour alone	52.0%	28.5%	51.2%	27.2%
pattern alone	50.3%	25.5%	50.0%	25.1%
structure alone	50.0%	25.0%	50.0%	25.0%
learned reference	72.7%	49.2%	84.6%	70.1%

904 clean PolyVore test outfits and 2,710 Fashionpedia test photos. A model fitted on the data puts ~0.95 on style and ~0.05 on colour. Colour, pattern and structure add no signal in the swap test: structure cannot by design (the swap keeps the category), and real outfits are not chosen for colour rules alone. They are kept because they give the user readable reasons and enforce wearable outfits; the trade-off is a team decision.



Swap-test AUC per part.



Outfits suggested from a mock wardrobe of 40 PolyVore test items.

6. Backend: FastAPI + MongoDB

backend/, run with `uvicorn app.main:create_app --factory`; OpenAPI docs at */docs*. Local MongoDB 4.0 (pymongo pinned below 4.14).

Endpoints	Purpose
<code>/auth/register</code> , <code>/auth/login</code> , <code>/me</code>	email + password (bcrypt), JWT; profile: name, min_coverage, language
<code>/items</code> (POST, GET, PATCH, DELETE), <code>/items/{id}/image</code>	upload → analysed (category, sub_category, pattern, colour, vector). PATCH stores user corrections in <i>corrected</i> ; the model's guesses stay in <i>predicted</i>
<code>/analyze</code>	shop candidate, not saved to the wardrobe, deleted after 24 h
<code>/buy-advice</code>	buy / think / skip for the last scan
<code>/outfits/score</code> , <code>/outfits/suggest</code> , <code>/outfits/complete</code> , <code>/outfits/limits</code>	the compatibility formula on the user's wardrobe; 422 on a clash; category limits for the Build page

Endpoints	Purpose
/similar, /catalog/{id}/image	look-alikes in the wardrobe, the public catalogue and the H&M shop ("buy something like this")
/chat, /chat/history	Gemini or the local model (CHAT_ENGINE) with function calling: list_wardrobe, suggest_outfits, score_outfit, buy_advice_last_scan
/admin/*	usage stats, model quality (how often users correct each field), user management, live editing of the formula

- **Clean photos before analysis** (*app/background.py*): every upload is turned upright with its EXIF rotation tag (phones store portrait photos sideways), shrunk to 1024 px, its background removed with rembg (U2-Net) and pasted on white, cropped around the item. When the item is worn, U2-Net keeps the whole person, so U2-Net cloth-seg cuts it down to the biggest garment. If the mask finds under 3% of the photo, the original is kept. ~3-5 s per photo on the CPU; *REMOVE_BACKGROUND=0* / *CLOTH_MODEL=* switch the steps off.
- **Privacy**: every query is filtered by *user_id*; another user's item answers 404.
- **Honest colour**: a colour below 0.7 confidence is left empty (the guess is kept in *predicted*) and the UI asks the user to confirm it.
- **H&M catalogue** replaces scraping (team decision 2026-10-03): 63k adult articles, pictures shrunk to 320 px in a private Kaggle notebook, embedded with FashionCLIP. The Zara / Bershka / Pull&Bear rows already scraped are kept only as a frozen demo.
- **Tests**: *python -m pytest* in *backend/* — 50 fast tests with fake models and a fake background remover on a *dressme_test* database, plus 2 real-model tests (*DRESSME_SLOW=1*).

7. Frontend: React + Tailwind

frontend/: React 19, Vite, TypeScript, Tailwind 4, talking to the API through the */api* dev proxy.

Screen	What the user does
Today	outfit of the day from their wardrobe, filtered by profile
Scan	photo in the friperie → analysis + buy / think / skip, with reasons
Wardrobe	add items by photo, confirm or correct the predicted labels
Build	pieces grouped by category; pick items, get a 0-100 score, or let the app complete the outfit (a clashing piece is swapped out)
Similar	"you already own something like this" and shop look-alikes
Chat	assistant that only talks about clothes the user owns
Profile	name, modesty level (<i>min_coverage</i>), language
Admin	stats, model quality, users, formula weights

- Three languages: English, French and Arabic (right-to-left), with plural rules.
- Design rules in *DESIGN.md*, product brief in *PRODUCT.md*.

8. Testing and demo readiness

Before the jury demo the whole app was run as a user would meet it: the real API, MongoDB and the React app in Chromium, at phone and desktop widths, with the test fakes standing in for the models and Gemini. Every screen and flow worked: scan → verdict → similar → add to wardrobe, item corrections, Build + complete, chat with tool calls, Arabic right-to-left, and the four admin pages. The walkthrough and the work that followed it found three bugs:

Found	Fix
Build page showed the score panel twice on phones	the desktop side panel is hidden below the desktop width
Phone portrait photos were analysed lying on their side (EXIF rotation ignored)	uploads apply the rotation tag; a test fails without the fix

Found	Fix
<i>merge_and_split.py</i> crashed on pandas 3 (allowed by <i>requirements.txt</i>)	one column type fixed; output byte-identical to pandas 2

DEMO.md holds the jury demo script: what to prepare the day before and 30 minutes before, a 9-step run order with talking points, and fallbacks. Everything except the Gemini chat runs offline (the local chat model, section 10, removes that exception). The real models and the chat still need a dry run on the demo laptop.

9. Local test photos

Our own phone photos (wardrobe and friperie) are the only data from the real DressMe setting, and the only way to measure traditional wear and swimwear. They are the **test set only**. The pipeline is ready; collection starts after the demo (targets: 500–1,000 wardrobe items, 150–300 friperie items, 50–100 traditional garments).

- *LOCAL_PHOTOS.md*: consent and privacy (anonymous contributor ids, no faces, shop owner's permission), how to shoot (one whole item per photo, plain surface, JPEG), and how to fill each label column.
- *src/map_local.py --init* lists new photos in *labels.csv*; the default run checks every label against *mappings/* (vocabularies, category ↔ sub_category, photo size, the same photo saved twice, ids that look like names) and stops with the line number, then writes *local.csv* (ids *lc_*).
- *merge_and_split.py* adds them to *dressme.csv* with *split* and *outfit_split* forced to test; their colours count as real labels.
- Still to do once photos exist: include *local* in the image cache, the embeddings and the evaluation scripts.

10. Local chat model

The free Gemini key allows only 20 requests a day, and one answer that uses tools costs 2 to 7: not enough for a demo or a user test. With *CHAT_ENGINE=ollama*, */chat* uses a model on our own machine (*app/chat_engine.py*, *OllamaEngine*) with the same system prompt and the same four tools. Ollama answers with tool calls, the backend runs them and sends the results back (at most 6 rounds). Details in *LLM.md*.

Step	What	Status
Base model	qwen3:4b-instruct in Ollama: best tool use of its size, no 'thinking' mode, knows French and Arabic	running
Dataset	<i>build_chat_dataset.py</i> : 3,000 / 200 / 300 synthetic chats (fr 45%, Darija 35% incl. Arabizi, en 20%) on random wardrobes; the backend's real tools produce the scores and reasons	built
Darija check	<i>reports/darija_review.md</i> : every Darija word and sentence with transliteration and meaning; initial team corrections are applied, full native review remains	in progress
Fine-tuning	QLoRA (Unsloth, rank 16, assistant turns only) on a free Kaggle T4, exported to GGUF: <i>dressme-chat</i>	next
Evaluation	<i>evaluate_chat.py</i> : base vs fine-tuned, per turn	baseline done

Evaluation on the synthetic test chats (75 decisions per model; each assistant turn is one decision, the model sees the real conversation up to that point):

Model	Tool decision	Tool name	Arguments	Language	s / decision
qwen3:4b-instruct	76.0%	33.3%	13.9%	94.9%	8.55

- **Base model**: it knows *when* to use a tool but rarely picks the right one with the right item ids (0% arguments on score and suggest). In real chats it scored made-up ids without listing the wardrobe first, gave a verdict without scoring, and invented reasons. Fine-tuning targets exactly these.
- **Speed** on the RTX 2050 with the backend's models loaded: ~21 tokens/s, 11-18 s per answer that uses tools (Ollama puts 58% of the model on the GPU, 42% on the CPU), ~35 s for the first answer (model loading).
- **Fixes from the first real run**: answers are capped at 600 tokens (the base model once repeated itself until a 5-minute timeout), and the assistant answers in the language the user writes in, not the profile language.

Unqualified suggestions return one best outfit; an explicit count is preserved.

11. Key decisions

Decision	Why
EfficientNet for category, sub_category and pattern	team rule "whichever is better": it won all three fields
Keep the gradient-boosting colour model	FashionCLIP vectors are worse on the hand labels
Colour empty below 0.7 confidence	a wrong colour hurts more than none; the user confirms it
Team-set compatibility weights	the data measures them but does not set them; style-only would lose readable reasons
One packed image cache	small-file reads were the training bottleneck
H&M catalogue instead of scraping	all three shop sites block scraping; we never try to get around bot protection
Remove the background of uploads	the models were trained on white product shots; a bed, rack or person in the photo confuses them
No repeated sub_category / category over its limit	a hard rule, not a lower score: two pairs of jeans is never an outfit
Local photos for test only	the one honest measure of the real setting; never trained on
Local chat model, fine-tuned on synthetic chats	no daily quota, works offline; the real tools write the answers, so the model learns our scores and never invented clothes
Predictions are never labels	<i>predicted_attributes.csv</i> and the <i>predicted</i> field stay separate from ground truth / user input

12. Limits and risks

- Traditional (jebba, kaftan) and swimwear are too rare in the public data to learn; traditional is read as dress. Local photos are needed.
- Street photos remain harder than product shots (category 93% vs 98%).
- Colour, pattern and structure add no measurable signal in the swap test; the team weights cost ~6 AUC points against style alone.
- PolyVore has no pattern labels, so its patterns are predictions.
- Background removal costs ~3-5 s per photo on the CPU; cloth-seg often misses items lying flat, so it is only used on worn items. Only the cleaned photo is kept, so a bad cut-out cannot be redone from the original.
- The models have not been measured on real phone photos yet (local test set pending).
- The H&M catalogue has no prices and is not Tunisian stock; the shop demo is frozen.
- The local chat model is slow on the 4 GB GPU (11-18 s per answer), and synthetic chats measure tool use and language, not how natural the answers sound. Initial Darija corrections are applied, but a complete native-speaker review is still pending.
- Datasets are for non-commercial academic use only and are never redistributed.

13. Next steps

- Dry run on the demo laptop with the real models, background removal and Gemini (*DEMO.md*).
- Collect and label local photos (*LOCAL_PHOTOS.md*), then evaluate the classifier, colour model and background removal on them.
- Team session to tune the compatibility weights and close the REVIEW rows.
- Local chat: native check of the Darija, fine-tune on Kaggle, then compare base vs *dressme-chat* and keep whichever is better (team rule).
- User test with the personas; use the admin "model quality" page (correction rates) to see where the models fail in real use.